

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
codec_id.py - ITU-T 음성 코덱(G.729 / G.723.1 ...) 구현체 식별 IDAPython 스크립트
```

```
=====
```

대상 : IDA 7.2 (IDAPython 2.7 및 3.x 모두), C/C++로 컴파일된 고정/부동소수점 구현

용도 : 바이너리(.ko / .so / ELF / 펌웨어)에 어떤 ITU 코덱의 어느 Annex/모드가

구현되어 있는지 상수 테이블·비트할당·심볼(디맹글 포함)로 추정한다.

설계 : 탐지 '엔진'과 코덱 '프로파일(데이터)'을 분리 -> 새 코덱은 CODECS 에

프로파일 dict 하나만 추가하면 된다(맨 아래 "코덱 추가 방법" 참고).

실행 : IDA > File > Script file... 로 이 파일을 로드. Output 창에 리포트 출력.

설정 : 아래 CONFIG 값(리네임/리포트 저장 등)만 바꾸면 된다.

주의 : 결과는 '후보'다. 최종 확정은 (1) 참조 코드 BinDiff/Diaphora 대조,

(2) ITU 테스트 벡터 bit-exact 검증으로 한다. 각 시그니처는 신뢰도

(spec / refcode / todo)를 달고 있으며, todo 는 ITU 배포 소스에서

실제 테이블 값을 채워 넣어야 하는 자리다.

```
=====
```

```
"""
```

```
from __future__ import print_function
```

```
import struct
```

```
import re
```

```
# -----
```

```
# CONFIG
```

```
# -----
```

```
CONFIG = {
```

```
    "rename_functions": False,    # True 면 시그니처로 특정한 함수에 이름을 붙인다(주의: 되돌리
```

```
    "rename_prefix":    "sig_",  # 리네임 시 접두사
```

```
    "write_report":     True,     # True 면 idb 옆에 codec_id_report.txt 저장
```

```
    "max_seg_bytes":    64 * 1024 * 1024,  # 세그먼트 1개 최대 스캔 크기(안전장치)
```

```
    "verbose":          True,     # 개별 히트 주소까지 출력
```

```
}
```

```
# -----
```

```
# IDA API 로딩 (7.2 호환, 버전차/미탑재 대비 전부 방어적)
```

```
# -----
```

```
try:
```

```
    import idaapi
```

```
    import idautils
```

```
    import idc
```

```
    import ida_bytes
```

```
    import ida_segment
```

```
    import ida_name
```

```

import ida_funcs
_IN_IDA = True
except Exception:
    _IN_IDA = False

def _msg(s):
    if _IN_IDA:
        idaapi.msg(s + "\n")
    else:
        print(s)

# -----
# 저수준 유틸: 세그먼트 바이트 캐시 / 엔디언 / 심볼 / xref
# -----
class Image(object):
    """스캔 대상 세그먼트들의 바이트를 한 번만 읽어 캐시한다."""

    def __init__(self):
        self.segs = []          # (start, end, bytes)
        self.total = 0
        self._load()

    def _load(self):
        for s in idautils.Segments():
            seg = ida_segment.getseg(s)
            if seg is None:
                continue
            size = seg.end_ea - seg.start_ea
            if size <= 0 or size > CONFIG["max_seg_bytes"]:
                continue
            data = ida_bytes.get_bytes(seg.start_ea, size)
            if not data:
                continue
            self.segs.append((seg.start_ea, seg.end_ea, data))
            self.total += len(data)

    def find_all(self, pattern):
        """bytes 패턴(정규식 컴파일 객체 또는 raw bytes)의 모든 시작 주소를 반환."""
        hits = []
        is_regex = hasattr(pattern, "finditer")
        for start, _end, data in self.segs:
            if is_regex:
                for m in pattern.finditer(data):
                    hits.append(start + m.start())
            else:

```

```

        i = data.find(pattern)
        while i != -1:
            hits.append(start + i)
            i = data.find(pattern, i + 1)
    return hits

def detect_endian():
    """리포트용 엔디언 추정('LE'/'BE'/'?'). 검색은 항상 양쪽 다 시도한다."""
    try:
        inf = idaapi.get_inf_structure()
        try:
            return "BE" if inf.is_be() else "LE"
        except Exception:
            return "BE" if getattr(inf, "mf", 0) else "LE"
    except Exception:
        return "?"

def proc_name():
    try:
        return idaapi.get_inf_structure().procname or "?"
    except Exception:
        return "?"

def demangle(name):
    """C++ 심볼을 디맹글. 실패 시 원본 반환."""
    if not name:
        return name
    try:
        dm = idc.demangle_name(name, idc.get_inf_attr(idc.INF_SHORT_DN))
        if dm:
            return dm
    except Exception:
        pass
    try:
        dm = ida_name.demangle_name(name, 0)
        if dm:
            return dm
    except Exception:
        pass
    return name

def iter_symbols():
    """(ea, raw_name, demangled_name) 를 순회."""

```

```

    for ea, raw in idutils.Names():
        yield ea, raw, demangle(raw)

def func_at(ea):
    """주소가 속한 함수의 (시작주소, 이름) 또는 (None, None)."""
    f = ida_funcs.get_func(ea)
    if f is None:
        return None, None
    return f.start_ea, ida_name.get_name(f.start_ea)

def xref_funcs(ea):
    """ea 를 참조하는 코드가 속한 함수 시작주소들의 집합."""
    out = set()
    for xr in idutils.XrefsTo(ea, 0):
        fs, _ = func_at(xr.frm)
        if fs is not None:
            out.add(fs)
    return out

# -----
# 시그니처 모델
# -----

def _pack_words(values, width, endian):
    # endian: "<" 또는 ">" (str)
    fmt = endian + ("h" if width == 16 else "i")
    return b"".join(struct.pack(fmt, v) for v in values)

def _regex_from_bytes(seq):
    """seq: int(0..255) 또는 None(와일드카드) 리스트 -> 컴파일된 bytes 정규식."""
    parts = []
    for b in seq:
        parts.append(b"." if b is None else re.escape(struct.pack("B", b)))
    return re.compile(b"".join(parts), re.DOTALL)

class Sig(object):
    """
    하나의 바이트 시그니처.
    feature : 이 시그니처가 입증하는 '특징' 이름 (코덱 프로파일에서 사용)
    weight  : 특징 가중치
    conf    : 'spec' (표준/문서 유도, 높음) | 'refcode' (참조코드 관례, 검증필요) | 'todo' (
        kind/values/width : words16|words32 (정수목록+폭) 또는 raw (바이트/None 목록)
    """

```

```

def __init__(self, feature, desc, weight=1.0, conf="spec",
              words16=None, words32=None, raw=None, both_endian=True):
    self.feature = feature
    self.desc = desc
    self.weight = weight
    self.conf = conf
    self.patterns = [] # (엔디언태그, 컴파일패턴)

    if raw is not None:
        self.patterns.append(("--", _regex_from_bytes(raw)))
        return
    vals, width = (words16, 16) if words16 is not None else (words32, 32)
    for endian, tag in ((" < ", "LE"), (" > ", "BE")):
        if not both_endian and tag == "BE":
            continue
        self.patterns.append((tag, _pack_words(vals, width, endian)))

def scan(self, img):
    """반환: [(엔디언태그, [주소...]), ...] (히트가 있는 것만)."""
    res = []
    for tag, pat in self.patterns:
        hits = img.find_all(pat)
        if hits:
            res.append((tag, hits))
    return res

# =====
# 코덱 프로파일 (데이터)
# =====
# 각 프로파일:
#   "signatures"    : Sig 목록
#   "symbols"       : (feature, weight, conf, 정규식패턴) -- raw/demangled 이름에 매칭
#   "strings"       : (feature, weight, 정규식패턴)         -- 세그먼트 바이트 문자열 매칭
#   "decide"        : function(feats, flags) -> (variant, confidence_note, evid
#   "present_feats" : 이 특징 중 하나라도 있으면 '코덱 존재'로 본다
# feat_set : 탐지된 feature 이름 집합.  flags: {"float":bool,"fixed":bool}

# ---- G.729 -----
G729_SIGS = [
    # 공통(모든 고정소수점 Annex) - 코덱 존재 확인
    Sig("g729.hpf_pre_b", "전처리 140Hz HPF 분자 b[] (Q12)", 3.0, "spec",
        words16=[1899, -3798, 1899]),
    Sig("g729.hpf_pre_a", "전처리 140Hz HPF 분모 a[] (Q12)", 3.0, "spec",
        words16=[4096, 7807, -3733]),
    Sig("g729.hpf_pre_a2", "전처리 140Hz HPF 분모 a[] 부호반전형", 2.0, "refcode",
        words16=[4096, -7807, 3733]),

```

```

Sig("g729.hpf_post_b", "후처리 100Hz HPF 분자 b[] (Q13)", 2.5, "spec",
    words16=[7699, -15398, 7699]),
Sig("g729.hpf_post_a", "후처리 100Hz HPF 분모 a[] (Q13)", 2.5, "spec",
    words16=[8192, 15836, -7667]),

# 8k 코어 비트할당 (본체/A/C 공통)
Sig("g729.bitalloc_8k", "8kbit/s 비트할당 bitsno(합80)", 2.0, "refcode",
    words16=[8, 10, 8, 1, 13, 4, 7, 5, 13, 4, 7]),

# 본체 vs Annex A 판별 (핵심)
Sig("g729.grid61", "본체 LP->LSP grid[61] 선두", 2.5, "spec",
    words16=[32760, 32723, 32588, 32365]),
Sig("g729.grid51", "Annex A LP->LSP grid[51] 선두", 2.5, "spec",
    words16=[32760, 32703, 32510, 32188]),

# Annex B (VAD/DTX/CNG)
Sig("g729.vad_a", "Annex B VAD 결정경계 a1..a4 (Table B.1)", 2.0, "spec",
    words16=[23488, -30504, -32768, 26214]),
Sig("g729.vad_b", "Annex B VAD 결정경계 b1..b4 (Table B.1)", 2.0, "spec",
    words16=[28521, 19446, -32768, -19661]),
Sig("g729.vad_thr", "Annex B VAD 임계 T1,T2 (Table B.1)", 1.5, "spec",
    words32=[671088640, 738197504]),
Sig("g729.sid_bits", "Annex B SID 비트할당 {1,5,4,5}(합15)", 1.5, "spec",
    words16=[1, 5, 4, 5]),

# Annex D (6.4k)
Sig("g729.d_track0", "Annex D 6.4k ACELP 트랙0 (Table D.2)", 2.0, "spec",
    words16=[1, 3, 6, 8, 11, 13, 16, 18]),
Sig("g729.d_track1", "Annex D 6.4k ACELP 트랙1 (Table D.2)", 1.5, "spec",
    words16=[0, 1, 2, 4, 5, 6, 7, 9, 10, 11, 12, 14]),

# 프레임손실 은닉 난수 LCG(본체/A) - imm 로도 검색(리터럴 풀)
Sig("g729.lcg", "프레임손실 은닉 LCG 상수 31821(리터럴)", 1.0, "spec",
    words32=[31821]),
]

G729_SYMBOLS = [
    ("g729.sym_main", 2.0, "spec", r"(?i)\b(Coder_ld8k|Decod_ld8k|Pitch_ol|Pitch_ol_fast|P
    ("g729.sym_a", 2.0, "spec", r"(?i)\b(Coder_ld8a|Decod_ld8a|Pitch_ol_fast|P
    ("g729.sym_b", 2.0, "refcode", r"(?i)\b(vad|Cod_cng|Dec_cng|Init_Cod_cng|Q
    ("g729.sym_d", 1.5, "refcode", r"(?i)(ld8kd|_g6k|phdisp|cor_h_x_e)"),
    ("g729.sym_e", 1.5, "refcode", r"(?i)(ld8e|bwfw|track_pi|acelp_e|calcgainc
    ("g729.sym_music", 1.5, "spec", r"(?i)(mus_dtct|music_?det|musdetect)"),
    ("g729.sym_integ", 1.0, "refcode", r"(?i)(ld8cp|ld8f|ld8g|ld8h|ld8i)"),
]

G729_STRINGS = [

```

```

("g729.str", 0.5, re.compile(rb"(?i)g\.?729")),
("g729.str2", 0.3, re.compile(rb"(?i)cs-?acelp")),
]

```

```

def g729_decide(feat, flags):
    ev = []
    has = lambda k: k in feat
    core_main = has("g729.grid61") or has("g729.sym_main")
    core_a = has("g729.grid51") or has("g729.sym_a")
    has_b = has("g729.vad_a") or has("g729.vad_b") or has("g729.vad_thr") or has(
    has_d = has("g729.d_track0") or has("g729.d_track1") or has("g729.sym_d")
    has_e = has("g729.sym_e")    # E 는 안전한 바이트 시그니처가 적어 심볼/후속 수동확인 위주
    has_music = has("g729.sym_music")
    is_float = flags["float"] and not flags["fixed"]

    if core_main:
        ev.append("본체 grid61/심볼")
    if core_a:
        ev.append("Annex A grid51/심볼")
    if has_b:
        ev.append("Annex B (VAD/DTX/CNG) 상수/심볼")
    if has_d:
        ev.append("Annex D (6.4k) 트랙/심볼")
    if has_e:
        ev.append("Annex E (11.8k) 심볼")
    if has_music:
        ev.append("음악검출 모듈")
    ev.append("부동소수점" if is_float else "고정소수점")

    # 통합형 우선 판정 (문서 6.8 판정표)
    if is_float and (has_d or has_e or has_b):
        return "Annex C+ (본체+B+D+E 부동소수점 통합)", "float+다중레이트", ev
    if has_d and has_e and has_b:
        return "Annex I (본체+B+D+E 통합)", "high", ev
    if has_d and has_e and not has_b:
        return "Annex H (본체+D+E, DTX 없음)", "high", ev
    if has_e and has_b:
        return "Annex G (본체+B+E)", "med", ev
    if has_d and has_b:
        return "Annex F (본체+B+D)", "med", ev
    if has_e:
        return "Annex E (11.8k)", "med", ev
    if has_d:
        return "Annex D (6.4k)", "high", ev
    if is_float:
        core = "Annex A 모드" if core_a and not core_main else ("본체 모드" if core_i

```

```

        return "Annex C (부동소수점, %s)" % core, "med", ev
    if has_b:
        base = "G.729AB" if (core_a and not core_main) else ("G.729B" if core_mai
        return "Annex B 포함 (%s)" % base, "high", ev
    if core_a and not core_main:
        return "Annex A (저복잡도 8k)", "high", ev
    if core_main and not core_a:
        return "G.729 본체 (8k)", "high", ev
    if core_main and core_a:
        return "본체+Annex A 동시 내장(예: 인코더 A + 디코더 본체)", "med", ev
    return "G.729 계열 흔적만 확인(코어 미특정)", "low", ev

# ---- G.723.1 -----
# 구조 상수(프레임/서브프레임/프레임크기)는 표준에서 직접 유도 -> conf=spec.
# LSP 코드북 등 값 테이블은 ITU 소스에서 채울 자리 -> conf=todo (아래 예시 형태만 제공).
G723_SIGS = [
    # 프레임 타입 2비트로 크기를 고르는 디스패치 테이블(가장 강력한 지문)
    # 6.3k=24B, 5.3k=20B, SID=4B, 미전송=1B (첫 바이트 하위 2비트: 0/1/2/3)
    Sig("g723.framesize", "프레임 크기표 {24,20,4,1} 바이트", 3.0, "spec",
        raw=[24, 20, 4, 1]),
    Sig("g723.framesize_w", "프레임 크기표(워드형) {24,20,4,1}", 2.0, "refcode",
        words16=[24, 20, 4, 1]),
    Sig("g723.framebits", "프레임 비트수 {192,160,32}", 2.0, "refcode",
        words16=[192, 160, 32]),

    # 구조 상수: 240샘플 프레임(0xF0), 60샘플 서브프레임(0x3C)
    Sig("g723.struct", "프레임=240, 서브프레임=60 인접 상수", 1.5, "spec",
        raw=[0xF0, 0x00, 0x3C, 0x00]),

    # — 아래는 ITU G.723.1 참조 소스에서 실제 값을 채워 넣을 자리(todo) —
    # 예: LSP 예측 코드북/후처리 계수 등. 값이 확정되면 conf 를 spec/refcode 로 올린다.
    # Sig("g723.lspcb", "LSP 코드북 선두(값 확정 필요)", 2.0, "todo", words16=[ ... ]),
    # Sig("g723.hpf", "입력 HPF 계수(값 확정 필요)", 2.0, "todo", words16=[ ... ]
]

G723_SYMBOLS = [
    ("g723.sym_core", 2.5, "refcode", r"(?i)\b(Comp_En|Estim_Pitch|Find_Acbk|Find
    ("g723.sym_mpmlq", 2.0, "refcode", r"(?i)(mp_?mlq|Find_MP_MLQ|Fcbk_?Mp)"), #
    ("g723.sym_acelp", 2.0, "refcode", r"(?i)(ACELP_?Lbc|Find_Acelp)"),
    ("g723.sym_vad", 2.0, "refcode", r"(?i)\b(Vad|Cng|Comp_?Vad|Update_?Cng|Swap
    ("g723.sym_generic", 1.0, "refcode", r"(?i)(g723|lbc\b|Coder_?723|Decod_?723)
]

G723_STRINGS = [
    ("g723.str", 0.6, re.compile(rb"(?i)g\.?723\.?1?")),
]

```



```

def g723_decide(feat, flags):
    ev = []
    has = lambda k: k in feat
    has_core = has("g723.framesize") or has("g723.framesize_w") or has("g723.fram
    has_63 = has("g723.sym_mpmlq")
    has_53 = has("g723.sym_acelp")
    has_a = has("g723.sym_vad")    # Annex A: 무음압축
    is_float = flags["float"] and not flags["fixed"]

    if has_core:
        ev.append("프레임 크기/구조 지문")
    if has_63:
        ev.append("MP-MLQ(6.3k) 심볼")
    if has_53:
        ev.append("ACELP(5.3k) 심볼")
    if has_a:
        ev.append("Annex A(VAD/DTX/CNG) 심볼")
    ev.append("부동소수점" if is_float else "고정소수점")

    rate = None
    if has_63 and has_53:
        rate = "듀얼레이트(5.3/6.3k)"
    elif has_63:
        rate = "6.3k(MP-MLQ) 전용"
    elif has_53:
        rate = "5.3k(ACELP) 전용"
    else:
        rate = "레이트 미특정(프레임표만 확인)"

    if is_float:
        return "G.723.1 Annex B(부동소수점) - %s" % rate, "med", ev
    if has_a:
        return "G.723.1 + Annex A(무음압축) - %s" % rate, "high" if has_core else "l
    if has_core:
        return "G.723.1 (고정소수점) - %s" % rate, "high" if (has_63 or has_53) else
    return "G.723.1 계열 흔적만 확인", "low", ev

# ---- 코덱 레지스트리 -----
CODECS = {
    "G.729": {
        "signatures": G729_SIGS,
        "symbols": G729_SYMBOLS,
        "strings": G729_STRINGS,
        "decide": g729_decide,

```

```

        "present":      ["g729.hpf_pre_b", "g729.hpf_pre_a", "g729.hpf_post_b",
                        "g729.bitalloc_8k", "g729.grid61", "g729.grid51",
                        "g729.sym_main", "g729.sym_a", "g729.str"],
    },
    "G.723.1": {
        "signatures": G723_SIGS,
        "symbols":     G723_SYMBOLS,
        "strings":     G723_STRINGS,
        "decide":      g723_decide,
        "present":     ["g723.framesize", "g723.framesize_w", "g723.framebits",
                        "g723.sym_core", "g723.sym_mpmlq", "g723.sym_acelp", "g723
    },
}

```

```

# -----
# 고정/부동소수점 및 기본연산자 휴리스틱
# -----
def detect_arith(img):
    """
    반환 {"float":bool,"fixed":bool,"notes":[...]}.
    - float : VFP/소프트플로트 심볼 존재
    - fixed : 포화 상수(0x7FFF/0x8000) 다수 + soft-float 부재
    """
    notes = []
    float_syms = re.compile(r"(?i)(__aeabi_[df](mul|add|sub|div|cmp)|__mulsf3|__a
                           r"__floatsi[sd]f|__fixsfsi|vmul\.f|vadd\.f)")
    is_float = False
    for _ea, raw, dm in iter_symbols():
        if float_syms.search(raw) or float_syms.search(dm):
            is_float = True
            notes.append("soft-float/VFP 심볼: %s" % (dm or raw))
            break

    # 포화 클램핑 상수 0x7FFF(32767) / 0x8000(-32768) 빈도
    sat_hi = img.find_all(_pack_words([32767], 16, "<")) + img.find_all(_pack_wor
    sat_lo = img.find_all(_pack_words([-32768], 16, "<"))
    has_fixed = (len(sat_hi) + len(sat_lo)) >= 8 and not is_float
    if has_fixed:
        notes.append("16비트 포화 상수 다수(0x7FFF/0x8000) -> 고정소수점 정황")
    return {"float": is_float, "fixed": has_fixed, "notes": notes}

# -----
# 탐지 실행
# -----
def run_signature_scan(img, sigs):

```

```

"""sigs 스캔 -> (feature_hits {feature:[(desc,conf,tag,addrs)]}, weight_by_fea
fh = {}
for sg in sigs:
    res = sg.scan(img)
    if not res:
        continue
    for tag, addrs in res:
        fh.setdefault(sg.feature, []).append((sg.desc, sg.conf, sg.weight, ta
return fh

def run_symbol_scan(symbols):
    """symbols 정의 스캔 -> {feature:[(name, ea), ...]}"""
    compiled = [(feat, w, conf, re.compile(rx)) for (feat, w, conf, rx) in symbol
hits = {}
for ea, raw, dm in iter_symbols():
    for feat, w, conf, rx in compiled:
        target = dm if dm else raw
        if rx.search(raw) or (dm and rx.search(dm)):
            hits.setdefault(feat, []).append((target, ea))
return hits

def run_string_scan(img, strings):
    hits = {}
    for feat, w, rx in strings:
        addrs = img.find_all(rx)
        if addrs:
            hits[feat] = addrs
    return hits

def maybe_rename(feature_hits):
    """RENAME 활성 시, 시그니처가 가리키는 테이블을 참조하는 함수에 이름 부여."""
    if not CONFIG["rename_functions"]:
        return
    for feat, entries in feature_hits.items():
        for (_desc, _conf, _w, _tag, addrs) in entries:
            for a in addrs[:1]: # 각 시그니처 첫 히트만
                for fs in xref_funcs(a):
                    nm = ida_name.get_name(fs)
                    if nm and (nm.startswith("sub_") or nm == ""):
                        newname = CONFIG["rename_prefix"] + feat.replace(".", "_")
                        try:
                            ida_name.set_name(fs, newname, ida_name.SN_NOWARN | i
                        except Exception:
                            pass

```

```

# -----
# 리포트
# -----

def _fmt_addrs(addr, n=3):
    show = ["0x%X" % a for a in addr[:n]]
    if len(addr) > n:
        show.append("... (%d)" % (len(addr) - n))
    return ", ".join(show)

def build_report(img, arith):
    lines = []
    def out(s=""):
        lines.append(s)

    out("=" * 74)
    out(" ITU 음성 코덱 식별 리포트 (codec_id.py)")
    out("=" * 74)
    out(" 프로세서: %-10s 엔디언(추정): %-3s 스캔 세그먼트: %d개 / %d bytes"
        % (proc_name(), detect_endian(), len(img.segs), img.total))
    out(" 연산방식: %s%s"
        % ("부동소수점" if arith["float"] else ("고정소수점" if arith["fixed"] else "마
        "" if not arith["notes"] else " [" + " ; ".join(arith["notes"][:2]) +
    out("-" * 74)

    any_codec = False
    for codec_name, prof in CODECS.items():
        sig_hits = run_signature_scan(img, prof["signatures"])
        sym_hits = run_symbol_scan(prof["symbols"])
        str_hits = run_string_scan(img, prof["strings"])

        feat = set(sig_hits.keys()) | set(sym_hits.keys()) | set(str_hits.keys())
        present = any(f in feat for f in prof["present"])
        if not present:
            continue
        any_codec = True

    # 커버리지 점수(가중치 기반, 참고용)
    total_w = sum(sg.weight for sg in prof["signatures"])
    got_w = sum(entries[0][2] for entries in sig_hits.values()) # feature당
    cov = (got_w / total_w * 100.0) if total_w else 0.0

    maybe_rename(sig_hits)

    variant, note, ev = prof["decide"](feat, arith)

```

```

out("")
out("### %s (시그니처 커버리지 %.0f%%)" % (codec_name, cov))
out(" ▶ 판정 : %s" % variant)
out(" ▶ 신뢰 : %s" % note)
out(" ▶ 근거 : %s" % (" , ".join(ev) if ev else "-"))

# 바이트 시그니처 히트
if sig_hits:
    out(" [바이트 시그니처]")
    for f, entries in sorted(sig_hits.items()):
        for (desc, conf, w, tag, addrs) in entries:
            mark = {"spec": "", "refcode": " (검증권장)", "todo": " (값미확정)"}
            line = " - %-34s %s x%d%s" % (desc, tag, len(addrs), mark)
            if CONFIG["verbose"]:
                line += " @ " + _fmt_addrs(addrs)
            out(line)

# 심볼 히트(디맹글)
if sym_hits:
    out(" [심볼]")
    for f, entries in sorted(sym_hits.items()):
        names = sorted(set(n for (n, _ea) in entries))
        out(" - %-18s %s" % (f, ", ".join(names[:6]) + (" ..." if len(

# 문자열
if str_hits:
    out(" [문자열]")
    for f, addrs in sorted(str_hits.items()):
        out(" - %-18s x%d @ %s" % (f, len(addrs), _fmt_addrs(addrs)))

if not any_codec:
    out("")
    out(" 등록된 어떤 코덱의 지문도 발견되지 않음.")
    out(" - 커스텀/비표준 구현, DSP 전용 프로세서(엔디언 워드주소), 또는 테이블 난독화 가능성.")
    out(" - todo 시그니처(G.723.1 값 테이블 등)를 ITU 소스로 채우면 탐지율이 올라간다.")

out("")
out("-" * 74)
out(" 주의: 위는 후보다. 확정은 (1) 참조코드 BinDiff/Diaphora 대조,")
out(" (2) ITU 테스트 벡터 bit-exact 검증으로 한다.")
out("-" * 74)
return "\n".join(lines)

# -----
# 진입점
# -----

def main():

```

```

if not _IN_IDA:
    print("이 스크립트는 IDA 안에서 실행해야 합니다 (File > Script file...).")
    print("등록된 코덱:", ", ".join(CODECS.keys()))
    return
img = Image()
if not img.segs:
    _msg("스캔할 세그먼트가 없습니다.")
    return
arith = detect_arith(img)
report = build_report(img, arith)
_msg(report)
if CONFIG["write_report"]:
    try:
        path = idc.get_idb_path()
        path = (path[:-4] if path.lower().endswith(".idb") or path.lower().en
        path = path + "_codec_id_report.txt"
        with open(path, "w") as f:
            f.write(report)
        _msg("[리포트 저장] %s" % path)
    except Exception as e:
        _msg("[리포트 저장 실패] %s" % e)

# =====
# 코덱 추가 방법 (요약)
# -----
# 1) Sig 목록을 만든다. 값은 참조 C 코드의 ROM 테이블/필터계수/비트할당에서
# 가져오고, 표준 문서에서 유도되면 conf="spec", 소스 관례면 "refcode",
# 아직 값을 못 채웠으면 "todo" 로 둔다(리포트에 표시됨).
# 2) 심볼 정규식 목록(디맹글된 C++ 이름에도 매칭됨)과 문자열 목록을 만든다.
# 3) decide(feats_set, flags) 함수를 작성한다. feats_set 은 탐지된 feature 이름
# 집합, flags 는 {"float":bool,"fixed":bool}. (variant, note, evidence[]) 반환.
# 4) CODECS["코덱명"] = {signatures, symbols, strings, decide, present} 등록.
# present 에는 "이것만 있어도 코덱 존재"로 볼 대표 feature 들을 넣는다.
# 값 테이블 시그니처는 최소 6~8 워드 이상으로 잡아야 오탐이 적다.
# =====

if __name__ == "__main__" or _IN_IDA:
    main()

```